

この dotfiles で学ぶ Nix

これは Nix を一般論だけで覚えるための資料ではない。このリポジトリを読み、少しずつ安全に変更できるようになるための手引きである。現在の構成は NixOS、Flakes、Home Manager を組み合わせて、2 台のホストと 1 人のユーザー環境を宣言的に管理している。

Nix の中心的な発想はシンプルだ。「今のマシンを手で変更する」のではなく、「あるべき状態をファイルに書き、そこから再現可能な世代を作る」。反映に失敗しても、NixOS では起動時に前の世代を選べる。とはいえ、ディスク UUID、ブート、認証のような設定は慎重に扱う。

最初に全体像をつかむ

flake.nix	入口：依存関係と出力を定義
├ hosts/desktop/configuration.nix	デスクトップ固有の NixOS 設定
│ └ modules/common.nix	全ホスト共通の小さな基盤パッケージ
│ └ modules/desktop.nix	GUI、音声、Wayland、systemd user service
├ hosts/thinkpad/configuration.nix	ThinkPad 固有の NixOS 設定
│ └ modules/common.nix	
│ └ modules/desktop.nix	
├ home/seli.nix	Home Manager の入口
│ └ home/common_user.nix	ユーザー用パッケージ、XDG 設定、user service
├ .config/	実際のアプリ設定 (Nix がリンクする)
└ flake_templates/	新規プロジェクト用の雛形

NixOS の設定と Home Manager の設定は似ているが、管理対象が違う。

- NixOS: root 権限が必要な OS 全体。カーネル、起動、ユーザー、サービス、デスクトップ基盤、etc、システム全体のパッケージを担当する。
- Home Manager: seli のホームディレクトリ。ユーザー用パッケージ、~/.config、シェル、ユーザー単位の systemd サービスを担当する。
- 通常の設定ファイル: .config/niri や .config/waybar のような、アプリ自身が読む設定。Home Manager が ~/.config/... からここへリンクを張る。

このリポジトリでは Home Manager は NixOS 構成の一部としても読み込まれ、同時に homeConfigurations.seli として単体でも利用できる。そのため、OS 全体に関わる設定は hosts/ または modules/ に、個人の環境だけに関わる設定は home/ に置くのが基本になる。

Flake: 依存関係と作れるものの目録

flake.nix はこのリポジトリの入口である。主に inputs、outputs、flake.lock を読む。

```
inputs = {
  nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";
  home-manager = {
    url = "github:nix-community/home-manager";
    inputs.nixpkgs.follows = "nixpkgs";
  };
};
```

inputs は外部のソースである。ここでは Nix パッケージ集合の nixpkgs、Home Manager、Firefox Nightly、独自パッケージを提供する複数の flake を受け取る。follows = "nixpkgs" は「この入力にも同じ nixpkgs を使わせる」という意味で、依存関係の重複を抑える。

flake.lock は入力を実際のコミットへ固定するロックファイルである。通常は手で編集しない。nix flake update または nix flake lock --update-input <名前> が更新する。更新はパッケージやモジュールの挙動を変えうるので、反映前に必ず評価・ビルドを確認する。

```
nixosConfigurations.desktop = nixpkgs.lib.nixosSystem {
  system = linuxSystem;
  specialArgs = { inherit inputs; };
  modules = [
```

```

    ./hosts/desktop/configuration.nix
    home-manager.nixosModules.home-manager
    { home-manager.users.seli = import ./home/seli.nix; }
  ];
};

```

outputs は「この flake が何を作れるか」の属性セット（名前付きの値の集合）である。このリポジトリでは次が重要。

- nixosConfigurations.desktop: デスクトップ機用の OS 構成。実際の hostname は selipaq。
- nixosConfigurations.thinkpad: ThinkPad 用の OS 構成。実際の hostname は selinoir。
- homeConfigurations.seli: Home Manager を単体で適用するためのユーザー構成。

specialArgs と home-manager.extraSpecialArgs に inputs を渡しているため、下位のファイルは inputs.wlmstr や inputs.firefox-nightly のように外部入力を参照できる。

Nix 言語の最小セット

Nix は設定専用の JSON ではなく、遅延評価される関数型言語である。ただし、このリポジトリでまず使う文法は少ない。

```

{ pkgs, lib, inputs, ... }: # 関数。必要な引数だけ受け取る
let
  name = "waybar";          # ローカル変数
  package = pkgs.waybar;
in
{
  environment.systemPackages = with pkgs; [ git waybar ];
  services.pipewire.enable = true;
  services.example.message = "hello ${name}";
}

```

- { ... } は属性セット。services.pipewire.enable = true; は入れ子の属性を定義する。
- [...] はリスト。パッケージやモジュールを並べる。
- ; は各定義の終端。# は行コメント、' ' ... ' ' は複数行文字列。
- { pkgs, ... }: は属性セットを引数に取る関数。NixOS/Home Manager が pkgs、lib、config など渡す。
- let ... in ... は補助変数を作る。modules/desktop.nix の waybarLaunch のような、長いスクリプトを名前にして再利用するために使う。
- \${...} は文字列展開。"\${pkgs.waybar}/bin/waybar" のように store 内の実行ファイルのパスを得る。
- with pkgs; [git ripgrep] は pkgs.git を git と短く書くための構文。小さなパッケージリストでは読みやすい一方、名前の出所が曖昧になるので、複雑な式で多用しない。
- inherit inputs; は inputs = inputs; の短縮形。

重要なのは「代入して上書きする言語」ではない点である。NixOS モジュールは多数のファイルが同じ option を宣言し、モジュールシステムが規則に従って一つの設定へ併合する。リストは多くの場合に連結され、単一の値が競合した場合は優先度が必要になる。

```

home.username = pkgs.lib.mkDefault "seli"; # 他の定義があればそちらを優先
environment.etc."crypttab".text = lib.mkForce ""; # 通常の定義より強く優先

```

このリポジトリにも上の両方がある。mkDefault は安全な既定値、mkForce は明確な理由がある競合解決に使う。mkForce は強力なので、まず既存の option 定義を検索してから使う。

実際の評価の流れ

sudo nixos-rebuild switch --flake .#desktop を実行すると、おおむね次の順に処理される。

```

flake.nix の nixosConfigurations.desktop
→ hosts/desktop/configuration.nix を読む

```

- imports された hardware-configuration.nix / common.nix / desktop.nix を読む
- Home Manager モジュールと home/seli.nix を読む
- 全モジュールの option を併合して評価する
- 必要な derivation をビルドまたはキャッシュから取得する
- 新しい system generation を作り、switch で有効化する

Nix は同じ入力・同じ設定なら同じ結果を再現できるように、ビルド結果を /nix/store に内容ハッシュ付きで置く。設定ファイルの参照も store にコピーされるのが通常だが、このリポジトリの mkOutOfStoreSymlink は例外で、作業ツリー内の .config へ直接リンクする。これはアプリ設定を即時に編集できる利点がある反面、Nix の世代だけではその内容を完全に固定しない、という意図的なトレードオフである。

ファイルごとの役割と、変更する場所

hosts/desktop と hosts/thinkpad

ここにはマシン固有の差分を置く。たとえば hostname、ファイルシステム UUID、LUKS、指紋認証、蓋を閉じた時の挙動、GPU 周辺の設定である。hardware-configuration.nix はインストーラが生成したハードウェア記述なので、原則として自動生成された内容を尊重する。

デスクトップと ThinkPad の共通部分は既に modules/common.nix と modules/desktop.nix に切り出されている。両方に同じ設定をコピーするより、共通モジュールへ移す方が将来の修正漏れを防げる。一方、UUID・スワップ・バッテリー・指紋センサーなどは共通モジュールへ入れない。

modules/common.nix

全ホストで共通の基盤 CLI パッケージを environment.systemPackages に入れている。ここに置くのは「ログインユーザーを問わず OS に常備したい最小限のコマンド」である。個人向けのエディタや GUI アプリを増やす場所としては、通常 home/common_user.nix の方が適切である。

modules/desktop.nix

デスクトップ基盤の共有設定である。GNOME/GDM、Hyprland、niri、PipeWire、ポータル、Bluetooth、フォント、仮想化などがここにある。さらに Waybar、swaync、cliphist、awww、swayidleなどを systemd.user.services として定義している。

この方針の要点は、コンポジタの起動コマンドへ常駐プロセスを直接足すのではなく、graphical-session.target に紐づく systemd user service として管理することだ。再起動方針、依存順序、ログ、終了時の片付けを systemd に任せられる。サービスを追加する時は、まず既存のサービスと同じ PartOf、After、wantedBy の形を参考にする。

home/common_user.nix と home/seli.nix

home/seli.nix は薄い入口で、common_user.nix を import し、SSH・フォント・ユーザー固有のパッケージを少し追加している。通常の変更先は home/common_user.nix である。

ここには home.packages、zsh、tmux、direnv、home.sessionVariables、xdg.configFile、ユーザー単位の systemd サービスがある。アプリを seli だけへ追加したいなら、packages = with pkgs; [...]; にパッケージ名を加える。たとえば jq は既にそこにあり、追加の形は次の通り。

```
packages = with pkgs; [  
  # 既存の項目...  
  imagemagick  
];
```

xdg.configFile の mkConfigLink は、たとえば "niri" を ~/.dotfiles/.config/niri へリンクする。よって Niri のキーバインドを変えるだけなら、Nix ファイルではなく .config/niri/config.kdl を編集する。新しい設定ディレクトリを Nix でリンク管理したいときは、既存と同じ形で 1 項目追加する。

```
"my-app" = {  
  source = mkConfigLink "my-app";
```

```
    recursive = false;
};
```

flake_templates/

flake_templates/rust.nix、typst.nix などは、プロジェクトごとの開発環境を始めるための雛形である。OS 設定とは別物で、各プロジェクト内に flake.nix として採用し、nix develop で必要なツールを一時的にシェルへ入れる用途に向く。

よく行う変更の実践

1. ユーザー用アプリを追加する

1. home/common_user.nix の packages に nixpkgs の属性名を加える。
2. まず評価する。
3. 問題がなければ Home Manager または NixOS 構成を反映する。

```
nix eval .#homeConfigurations.seli.activationPackage.drvPath
home-manager switch --flake .#seli
```

このマシンで NixOS 経由の Home Manager を使うなら、後述の nixos-rebuild switch を使って OS 設定と一緒に反映してもよい。パッケージ名が分からないときは、まず nix search nixpkgs <名前> で候補を確認する。

2. システムサービスを有効にする

サービスが全ホスト共通なら modules/desktop.nix、機種固有なら対応する hosts/<機種>/configuration.nix を編集する。

```
services.example.enable = true;
```

option の正確な名前と型は NixOS Options Search で調べる。既存の services.tailscale.enable、services.openssh.enable、services.fprintd.enable はこのリポジトリで読める実例である。設定後は対象ホストの構成をビルドする。

3. Wayland アプリの設定を変える

Niri、Hyprland、Waybar、swaync、Noctalia の見た目やキーバインドは主に .config/ 以下にある。home/common_user.nix が out-of-store symlink を作るため、リンクが既に有効なら多くの場合はアプリの再読み込みまたは再起動だけでよい。新規リンクを追加した場合は Home Manager を反映する。

4. 2 台共通の設定にする

両方の host が imports している modules/common.nix または modules/desktop.nix に移す。移した後は両構成を評価する。片方だけに適用したい値を共通モジュールへ置くと、特にストレージ・電源・入力デバイスで事故になりやすい。

安全な確認と反映の順番

作業ディレクトリはこのリポジトリのルートであることを前提にする。switch は実際に稼働中のシステムを切り替えるので、最初は評価とビルドを優先する。

```
# 変更内容と入力の状態を確認
```

```
git diff
```

```
nix flake metadata
```

```
# flake の評価チェック（ビルドはしない）
```

```
nix flake check --no-build
```

```
# 現在のホスト向けにビルドだけする
```

```
sudo nixos-rebuild build --flake .#desktop
```

```
# ThinkPad では .#thinkpad を使う
```

```
# 結果を有効化する。成功後に新しい generation へ切り替わる
sudo nixos-rebuild switch --flake .#desktop
```

build は生成物を作るが、稼働中の設定を切り替えない。test はその場で有効化するが、次回起動時の既定にはしない。switch は有効化し、次回起動にも採用する。ネットワーク、ログイン、ブート周りを触るときは、ローカルの別 TTY や起動メニューで戻れる状態を保ってから switch する。

Home Manager だけを単体で確認・反映する場合は次を使う。

```
nix build .#homeConfigurations.seli.activationPackage
home-manager switch --flake .#seli
```

ただし通常の NixOS 運用では、flake.nix が NixOS モジュール内に同じ Home Manager 設定を組み込んでいるため、OS の nixos-rebuild だけで一貫して適用できる。

調査の工具箱

```
# option の最終値と、その定義元を追う（例）
nixos-option services.pipewire.enable
nixos-option environment.systemPackages
```

```
# 現在と過去の NixOS generation
sudo nix-env --list-generations --profile /nix/var/nix/profiles/system
```

```
# user service の状態とログ
systemctl --user status noctalia.service
journalctl --user -u swayidle.service -b
```

```
# パッケージ候補の検索
nix search nixpkgs firefox
```

```
# Nix 式を試す。`nix repl` では :lf . でこの flake を読み込める
nix repl
```

nixos-option は「最終的に何が設定されたか」を見るのに有用だが、現在起動している世代を対象にする。編集直後の結果を検査したい場合は、まず nixos-rebuild build や nix flake check を行う。

更新、世代、ロールバック

入力更新は意図的に行う。特定の入力だけを更新する例は次の通り。

```
nix flake lock --update-input nixpkgs
git diff --flake.lock
nix flake check --no-build
sudo nixos-rebuild build --flake .#desktop
```

更新後に問題が出たら、まだコミットしていない変更なら flake.lock の差分を確認し、元に戻す判断ができる。すでに switch して問題が出た場合は、起動メニューで以前の generation を選ぶか、CLI で前の世代へ戻す。

```
sudo nixos-rebuild switch --rollback
```

世代はガベージコレクションが消すまで残る。このリポジトリでは NixOS の nix.gc が毎日動き、7 日より古いものを削除する設定になっている。大きな変更を試す前は、直近の正常な generation が残っていることを確認する。

この構成での判断基準

- ・「root 権限・起動時・全ユーザーに関係するか？」なら hosts/ または modules/。
- ・「seli のホームディレクトリ・個人のアプリ・XDG 設定か？」なら home/。
- ・「アプリ固有の設定値・見た目・キーバインドか？」なら .config/。
- ・「2 台で共通か？」ならモジュール化を検討し、「ハードウェアに結び付くか？」なら host に残す。
- ・「何を変えたか不明なまま更新しない」。git diff、評価、build、switch の順に小さく進める。

最初の練習としては、`home/common_user.nix` に使いたい CLI パッケージを一つ追加し、`nix flake check --no-build`、`nix build .#homeConfigurations.seli.activationPackage`、反映の順に試すのがおすすめである。次に `.config/waybar` の変更と `user service` の再起動を試すと、Nix が管理する部分と直接編集する部分の境界が自然に分かるようになる。